

수업 02 · 2026-10-14

Python 기초

짧은 Python 코드를 읽고 실행합니다. 변수, 조건, 반복을 이용해 센서 기록을 점검해 봅니다.

10:00~17:00 · 50분 수업 후 10분 휴식 · 중식 11:50~13:00

오늘 필수 활동 10개 · 교시별 설명과 예제를 보고 직접 해 봅니다.

Python 기초 · 값과 실행 흐름

2026-10-14 · Python 기초

짧은 Python 코드를 읽고 실행합니다. 변수, 조건, 반복을 이용해 센서 기록을 점검해 봅니다.

수업은 50분씩 6교시입니다. 중식은 11:50~13:00입니다. 원본은 그대로 두고 work 폴더의 파일에서 안내한 부분을 고칩니다. 계정이나 실제 장비 없이 연습용 자료를 사용합니다.

오늘의 연결

지난 데이터 수업에서 읽은 온도와 식별자를 오늘은 Python 코드로 다뤄 봅니다.

먼저 구별할 화면

- 파일 탐색기는 저장한 파일의 위치를 찾습니다.
- 텍스트 편집기는 코드와 데이터를 읽고 고칩니다.
- 터미널은 명령을 입력하고 실제 실행 결과를 확인합니다.
- 웹 교안은 설명·원문·출력 예를 제공합니다. Python 실행기는 아닙니다.

참여 순서

1. 압축을 일반 폴더에 풀고 data·code·work의 위치를 유지합니다.
2. lesson의 개념을 읽고 activity-guide의 해당 활동 입력을 엽니다.
3. work의 답칸에 예상과 이유를 먼저 쓰고 지정한 값을 수정합니다.
4. 저장한 work 파일을 실행합니다. 실행할 수 없으면 예상한 결과를 강사의 출력이나 제공된 예와 비교합니다.
5. 확인 방식을 기록합니다. 개인 실행을 하지 않았으면 실행 성공으로 적지 않습니다.

도구를 준비하지 못했을 때

설치가 안 되면 제공된 코드와 출력으로 수업을 따라갑니다. 같은 입력값을 넣고 한 줄씩 처리한 뒤의 값을 종이에 적어 봅니다. 강사가 실행 환경을 준비하면 직접 실행해 확인합니다.

[교안](#) · [실습](#) · [용어](#)

Windows · 처음 실행할 위치 만들기

다른 OS는 강사의 해당 환경 안내를 따릅니다. 설치 문제는 개념 평가와 따로 기록합니다.

1. 예제 ZIP을 우클릭 → 모두 추출 → 추출합니다. ZIP 안을 보는 창에서 실행하지 않습니다.
2. 탐색기로 압축을 푼 곳의 course 폴더를 엽니다. 안에 data, code, work 세 폴더가 실제로 보이는지 확인합니다. 폴더가 한 겹 더 있으면 그 안으로 들어갑니다.

3. 탐색기 위 주소 표시줄을 클릭해 **powershell**을 입력하고 Enter를 누릅니다. 현재 폴더에서 PowerShell 창이 열립니다. 편집기에서 파일을 연 것만으로 실행 위치가 정해지지는 않습니다.

4. 아래 명령을 한 줄씩 입력하고 Enter를 누릅니다. 앞의 **PS ...>** 문구는 입력하지 않습니다.

```
Get-Location
Get-ChildItem -Directory
Test-Path .\work\A02_print.py
```

Get-Location은 지금 실행할 폴더의 실제 경로를 보여 줍니다. 끝이 **course**인지 확인합니다. **Get-ChildItem** 목록에는 **data**, **code**, **work**가 보여야 합니다. 마지막 명령은 파일이 있는지 확인하며 정상 결과는 **True**입니다. 내 실제 경로: ____ / 확인 결과: ____.

1. **work/A02_print.py** 첫 print 문구를 **오늘의 점검 시작**으로 고치고 저장합니다. 같은 PowerShell 창에서 실행합니다.

```
python -X utf8 work/A02_print.py
```

예상 두 줄은 다음과 같습니다.

```
오늘의 점검 시작
S01 22.5
```

- **False, can't open file, No such file:** 실제 폴더를 탐색기로 다시 열고 2~4단계를 반복합니다. 파일 이름이 **.py.txt**가 아닌지 탐색기의 **보기 → 표시 → 파일 확장명**에서 확인합니다.

- 첫 줄이 **Python 점검 시작:** 수정 파일을 저장했는지, 실행 명령이 **code/**가 아닌 **work/**를 가리키는지 확인합니다.

- **python**을 찾지 못하거나 Store 안내가 뜬: 강사에게 설치 상태를 확인받습니다. 설치는 강사와 확인하고 예상한 결과를 제공된 출력과 비교하며 **개인 실행 미완료**를 적습니다.

실행 위치 확인 → 파일 저장 → 명령 입력 → 실제 출력 기록을 서로 다른 행동으로 확인합니다.

수업 전, 이 세 가지부터 확인하세요

앞선 수업을 기억하지 못해도 아래 질문과 풀이부터 다시 읽으면 됩니다. 먼저 답을 적어 보고 풀이와 비교하세요.

데이터와 명령

질문: 22.5와 **print(22.5)**는 무엇이 다른가요?

풀이: 22.5는 값입니다. **print(22.5)**는 그 값을 화면에 표시하라는 호출입니다. 저장된 명령을 실행해야 출력이 생깁니다.

따옴표

질문: "22.5" + "1"의 결과가 23.5인가요?

풀이: 문자열을 잇는 결과는 "22.51"입니다. 계산할 숫자인지 식별자인지 먼저 결정하고 자료형을 확인합니다.

분기

질문: 온도가 기준과 정확히 같으면 > 조건은 참인가요?

풀이: 거짓입니다. 경계값을 직접 비교해야 >와 >=의 업무 의미 차이를 이해할 수 있습니다.

오늘의 수업과 복습을 나누는 방법

수업은 10:0010:50, 11:0011:50, 13:0013:50, 14:0014:50, 15:0015:50, 16:0016:50입니다. 중식은 11:5013:00이며 수업 사이에는 10분 쉽니다. 마지막 16:5017:00은 휴식과 저장·퇴실 준비로 배치했습니다.

교시	수업에서 먼저 확인할 활동	추가·복습
1	A01, A02	A03
2	A04, A05, A06	새 입력으로 한 번 더 확인
3	A07, A08, A09	새 입력으로 한 번 더 확인
4	A10, A11, A12	새 입력으로 한 번 더 확인
5	A13, A14, A15	새 입력으로 한 번 더 확인
6	A16, A17, A18	새 입력으로 한 번 더 확인

교안의 사례와 풀이를 읽고 필수 활동을 해 봅니다. 결과를 확인했으면 추가 문제로 복습하세요. 모든 페이지를 수업 시간 안에 마칠 필요는 없습니다. 실행이 안 되면 제공된 출력과 비교하며 코드의 처리 순서를 적어 봅니다. 직접 실행했는지는 따로 표시합니다.

2026-10-14 · Python 기초

짧은 Python 코드를 읽고 실행합니다. 변수, 조건, 반복을 이용해 센서 기록을 점검해 봅니다.

****확정 운영은 50분×6교시·순수 수업 300분이며 중식은 11:50~13:00입니다. 계정이나 실제 장비 없이 연습용 자료를 사용합니다.**

1교시 · 파일에서 출력까지

10:00~10:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

A02 · 첫 출력과 주석

개념. print는 전달받은 값을 화면에 보여 주는 기본 함수입니다. 함수라는 말은 우선 이름 붙은 일을 호출하는 도구로 이해합니다. 괄호 안에 보여 줄 내용을 넣고, 문자열은 따옴표로 감쌉니다.

왜 필요한가. 짧은 출력을 직접 바꾸면 편집한 내용이 실행에 반영되는 과정을 볼 수 있습니다. 복잡한 프로그램을 만들기 전에 실행 통로를 확인하는 작은 성공 기준입니다.

함께 보는 예. 첫 줄의 # 뒤 문장은 사람이 읽는 설명인 주석입니다. Python은 이 주석을 실행하지 않습니다. 따옴표 안의 #는 글자의 일부이므로 주석이 아니라는 점도 비교합니다.

헛갈리는 경우. print라는 이름을 Print로 쓰면 같은 도구가 아닙니다. 큰따옴표는 키보드로 입력하는 곧은 따옴표이며 문서 편집기의 장식 따옴표로 바꾸면 문법 문제가 생길 수 있습니다.

전체 코드

```
# 화면에 수업 시작을 표시합니다.
print("Python 점검 시작")
print("S01", 22.5)
```

각 줄의 뜻

줄	코드	뜻
1	# 화면에 수업 시작을 표시합니다.	사람을 위한 설명이며 실행 결과를 만들지 않습니다.
2	print("Python 점검 시작")	문자열 한 개를 화면에 보여 줍니다.
3	print("S01", 22.5)	문자열과 숫자를 쉼표로 나누어 한 줄에 출력합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A02_print.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A02_print.py
```

원본 코드의 예상 출력:

```
Python 점검 시작
S01 22.5
```

작은 수정: Python 점검 시작을 **오늘의 점검 시작**로 바꿉니다.

수정 후 예상 출력:

```
오늘의 점검 시작
S01 22.5
```

학생 적용

1. work/A02_print.py를 텍스트 편집기로 엽니다.
2. 수정 전 출력 두 줄을 예상해 노트에 씁니다.
3. 첫 안내 문구를 '오늘의 점검 시작'으로 바꾸고 저장합니다.

4. course 폴더의 터미널에서 `python -X utf8 work/A02_print.py`를 실행하고 예상과 대조합니다.

예상 결과: 첫 안내 문구만 바뀌며 둘째 줄 S01 22.5는 유지됩니다.

확인 질문: 주석의 문구만 바꾸면 출력은 달라질까요?

막혔을 때: python을 찾지 못하면 설치를 혼자 진행하지 말고 강사에게 환경을 확인합니다. 그동안 제공된 출력과 수정안을 비교하고 개인 실행 미완료를 표시합니다.

[활동 자료와 작성칸](#)에서 A02를 엽니다.

Windows · 처음 실행할 위치 만들기

다른 OS는 강사의 해당 환경 안내를 따릅니다. 설치 문제는 개념 평가와 따로 기록합니다.

1. 예제 ZIP을 우클릭 → **모두 추출** → **추출**합니다. ZIP 안을 보는 창에서 실행하지 않습니다.
2. 탐색기로 압축을 푼 곳의 **course** 폴더를 엽니다. 안에 **data**, **code**, **work** 세 폴더가 실제로 보이는지 확인합니다. 폴더가 한 겹 더 있으면 그 안으로 들어갑니다.
3. 탐색기 위 주소 표시줄을 클릭해 **powershell**을 입력하고 Enter를 누릅니다. 현재 폴더에서 PowerShell 창이 열립니다. 편집기에서 파일을 연 것만으로 실행 위치가 정해지지 않습니다.
4. 아래 명령을 한 줄씩 입력하고 Enter를 누릅니다. 앞의 **PS ...>** 문구는 입력하지 않습니다.

```
Get-Location
Get-ChildItem -Directory
Test-Path .\work\A02_print.py
```

Get-Location은 지금 실행할 폴더의 실제 경로를 보여 줍니다. 끝이 **course**인지 확인합니다. **Get-ChildItem** 목록에는 **data**, **code**, **work**가 보여야 합니다. 마지막 명령은 파일이 있는지 확인하며 정상 결과는 **True**입니다. 내 실제 경로: ____ / 확인 결과: ____.

1. **work/A02_print.py** 첫 print 문구를 **오늘의 점검 시작**으로 고치고 저장합니다. 같은 PowerShell 창에서 실행합니다.

```
python -X utf8 work/A02_print.py
```

예상 두 줄은 다음과 같습니다.

```
오늘의 점검 시작
S01 22.5
```

- **False, can't open file, No such file:** 실제 폴더를 탐색기로 다시 열고 2~4단계를 반복합니다. 파일 이름이 **.py.txt**가 아닌지 탐색기의 **보기** → **표시** → **파일 확장명**에서 확인합니다.

- 첫 줄이 **Python 점검 시작:** 수정 파일을 저장했는지, 실행 명령이 **code/**가 아닌 **work/**를 가리키는지 확인합니다.

- **python**을 찾지 못하거나 Store 안내가 뜬: 강사에게 설치 상태를 확인받습니다. 설치는 강사와 확인하고 예상한 결과를 제공된 출력과 비교하며 **개인 실행 미완료**를 적습니다.

실행 위치 확인 → 파일 저장 → 명령 입력 → 실제 출력 기록을 서로 다른 행동으로 확인합니다.

A02 · 직접 해보기

A02 · 첫 출력과 주석

목적: 짧은 출력을 직접 바꾸면 편집한 내용이 실행에 반영되는 과정을 볼 수 있습니다. 복잡한 프로그램을 만들기 전에 실행 통로를 확인하는 작은 성공 기준입니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A02_input.md](#)

- [A02_print.py](#)

- [A02_notes.md](#)

- [A02_print.py](#)

단계

1. work/A02_print.py를 텍스트 편집기로 엽니다.
2. 수정 전 출력 두 줄을 예상해 노트에 씁니다.
3. 첫 안내 문구를 '오늘의 점검 시작'으로 바꾸고 저장합니다.
4. course 폴더의 터미널에서 `python -X utf8 work/A02_print.py`를 실행하고 예상과 대조합니다.

예상 결과: 첫 안내 문구만 바뀌며 둘째 줄 S01 22.5는 유지됩니다.

확인 방법: 주석의 문구만 바꾸면 출력은 달라질까요?

오류 해결: python을 찾지 못하면 설치를 혼자 진행하지 말고 강사에게 환경을 확인합니다. 그동안 제공된 출력과 수정안을 비교하고 개인 실행 미완료를 표시합니다.

Windows · 처음 실행할 위치 만들기

다른 OS는 강사의 해당 환경 안내를 따릅니다. 설치 문제는 개념 평가와 따로 기록합니다.

1. 예제 ZIP을 우클릭 → **모두 추출** → **추출**합니다. ZIP 안을 보는 창에서 실행하지 않습니다.
2. 탐색기로 압축을 푼 곳의 **course** 폴더를 엽니다. 안에 **data**, **code**, **work** 세 폴더가 실제로 보이는지 확인합니다. 폴더가 한 겹 더 있으면 그 안으로 들어갑니다.
3. 탐색기 위 주소 표시줄을 클릭해 **powershell**을 입력하고 Enter를 누릅니다. 현재 폴더에서 PowerShell 창이 열립니다. 편집기에서 파일을 연 것만으로 실행 위치가 정해지지는 않습니다.
4. 아래 명령을 한 줄씩 입력하고 Enter를 누릅니다. 앞의 **PS ...>** 문구는 입력하지 않습니다.

```
Get-Location
Get-ChildItem -Directory
Test-Path .\work\A02_print.py
```

Get-Location은 지금 실행할 폴더의 실제 경로를 보여 줍니다. 끝이 **course**인지 확인합니다. Get-ChildItem 목록에는 **data, code, work**가 보여야 합니다. 마지막 명령은 파일이 있는지 확인하며 정상 결과는 **True**입니다. 내 실제 경로: ____ / 확인 결과: ____.

1. **work/A02_print.py** 첫 print 문구를 **오늘의 점검 시작**으로 고치고 저장합니다. 같은 PowerShell 창에서 실행합니다.

```
python -X utf8 work/A02_print.py
```

예상 두 줄은 다음과 같습니다.

```
오늘의 점검 시작
S01 22.5
```

- **False, can't open file, No such file:** 실제 폴더를 탐색기로 다시 열고 2~4단계를 반복합니다. 파일 이름이 **.py.txt**가 아닌지 탐색기의 **보기 → 표시 → 파일 확장명**에서 확인합니다.

- 첫 줄이 **Python 점검 시작:** 수정 파일을 저장했는지, 실행 명령이 **code/**가 아닌 **work/**를 가리키는지 확인합니다.

- **python**을 찾지 못하거나 Store 안내가 뜸: 강사에게 설치 상태를 확인받습니다. 설치는 강사와 확인하고 예상한 결과를 제공된 출력과 비교하며 **개인 실행 미완료**를 적습니다.

실행 위치 확인 → 파일 저장 → 명령 입력 → 실제 출력 기록을 서로 다른 행동으로 확인합니다.

수정 전 예상: _____
 확인한 결과와 이유: _____

A03 · 저장·실행 순서와 오류 단서

개념. 편집기의 변경 내용은 저장하기 전까지 파일에 반영되지 않을 수 있습니다. 실행 명령은 디스크에 저장된 특정 파일을 읽습니다. 터미널은 명령을 입력하고 실행 결과를 보는 창이며 코드 편집기와 역할이 다릅니다.

왜 필요한가. 같은 이름의 원본과 작업본이 있을 때 다른 파일을 실행하면 수정한 문구가 나오지 않습니다. 재실행을 반복하기 전에 파일 경로와 저장 여부를 확인하면 원인을 좁힐 수 있습니다.

함께 보는 예. 작업본에는 새 문구가 보이지만 **code/A02_print.py**를 실행했다면 원본 문구가 나오는 것이 자연스럽습니다. 명령 끝의 경로를 **work/A02_print.py**로 맞춰야 작업본을 실행합니다.

헛갈리는 경우. **SyntaxError**는 Python이 코드를 문장으로 읽지 못했다는 단서입니다. 화면 전체를 지우거나 Python을 재설치하기 전에 마지막 오류 종류와 해당 줄, 바로 앞 줄의 따옴표·괄호를 확인합니다.

제공 입력

```
사례 1: 편집기 탭에는 work/A02_print.py, 터미널 명령은 python -X utf8 code/A02_print.py.
사례 2: print("점검 시작") → SyntaxError.
사례 3: print("점검 시작")를 고쳤지만 저장 표시가 남아 있음.
```

학생 적용

1. 세 사례에서 보이는 사실과 추정 원인을 다른 칸에 적습니다.
2. 각 사례의 다음 확인 동작을 하나만 고릅니다.

3. 잘못된 따옴표 코드를 종이에 고친 뒤 괄호와 따옴표의 짝을 표시합니다.

4. 작업본 저장 후 동일 경로를 실행하는 순서를 네 단계로 작성합니다.

예상 결과: 다른 파일 실행·닫는 따옴표 누락·미저장 가능성을 각각 구별합니다.

확인 질문: 오류 줄 번호가 마지막 줄을 가리키면 그 줄만 보면 충분할까요?

막혔을 때: 오류 메시지를 해석하기 어렵다면 명령, 실행 파일 경로, 오류 마지막 줄을 그대로 기록해 강사에게 보여 줍니다.

비슷한 문구로 요약해 단서를 잃지 않습니다.

[활동 자료와 작성칸](#)에서 A03를 엽니다.

A03 · 직접 해보기

A03 · 저장·실행 순서와 오류 단서

목적: 같은 이름의 원본과 작업본이 있을 때 다른 파일을 실행하면 수정한 문구가 나오지 않습니다. 재실행을 반복하기 전에 파일 경로와 저장 여부를 확인하면 원인을 좁힐 수 있습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A03_input.md](#)

- [A03_notes.md](#)

단계

1. 세 사례에서 보이는 사실과 추정 원인을 다른 칸에 적습니다.

2. 각 사례의 다음 확인 동작을 하나만 고릅니다.

3. 잘못된 따옴표 코드를 종이에 고친 뒤 괄호와 따옴표의 짝을 표시합니다.

4. 작업본 저장 후 동일 경로를 실행하는 순서를 네 단계로 작성합니다.

예상 결과: 다른 파일 실행·닫는 따옴표 누락·미저장 가능성을 각각 구별합니다.

확인 방법: 오류 줄 번호가 마지막 줄을 가리키면 그 줄만 보면 충분할까요?

오류 해결: 오류 메시지를 해석하기 어렵다면 명령, 실행 파일 경로, 오류 마지막 줄을 그대로 기록해 강사에게 보여 줍니다.

비슷한 문구로 요약해 단서를 잃지 않습니다.

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 입력·처리·출력과 코드 파일

[이 교시의 다른 예와 코드 해설 열기](#)

2교시 · 값에 이름 붙이기

11:00~11:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

A04 · 변수는 값에 붙인 이름

사례로 이해하기 · 재고 갱신을 한 줄씩 추적하기

먼저 강사의 예를 보고 같은 순서로 함께 확인합니다. 추가 문제는 필수 활동을 마친 뒤 풀어 보세요.

창고에서 새 물품이 들어오고 일부가 나갑니다. 담당자는 '처음 8개, 입고 3개, 출고 2개'를 순서대로 기록합니다. Python에서는 현재 재고에 stock이라는 이름을 붙이고 각 작업이 끝날 때의 값을 연결합니다. 변수는 여러 값을 자동으로 보관하는 이력표가 아닙니다. 같은 이름에 새 값을 대입하면 그 시점의 값이 바뀝니다. 과거 기록이 필요하다면 별도의 기록을 남겨야 합니다. 먼저 종이에 실행 줄·현재 값·출력 세 열을 만듭니다.

이 사례의 완결 코드는 다음과 같습니다. work 폴더에 새 파일 E02_stock.py를 UTF-8로 저장하고 course 폴더의 터미널에서 `python -X utf8 work/E02_stock.py`를 실행합니다. 기존 파일을 덮어쓰지 않습니다.

```
stock = 8
stock = stock + 3
print(stock)
stock = stock - 2
print(stock)
```

첫 줄은 stock에 정수 8을 연결합니다. 둘째 줄은 오른쪽의 현재 stock 8에 3을 더해 11을 만든 다음 왼쪽 이름에 연결합니다. 셋째 줄은 그때의 11을 화면에 표시합니다. 넷째 줄은 현재 11에서 2를 빼 9로 갱신합니다. 마지막 줄은 9를 출력합니다. 따라서 예상 출력은 첫 줄 11, 둘째 줄 9입니다. =는 수학의 양쪽이 같다는 설명이 아니라 오른쪽 계산 결과를 왼쪽 이름에 연결하는 대입 기호입니다.

입고와 출고를 모두 처리한 뒤 출력 두 줄을 적는 것이 아니라 print를 만나는 순간의 값을 기록합니다. 뒤의 갱신이 이미 나온 첫 출력을 고치지는 않습니다. NameError이면 stock 철자와 대소문자, 처음 대입한 줄을 확인합니다. can't open file 이면 작업 파일 저장 위치를 먼저 확인합니다. 실행 환경이 없으면 같은 추적표를 쓰고 강사 실행과 비교했다는 표시를 남깁니다.

혼자 풀어 볼 문제

복습·도전: 물 12병을 준비하고 4병을 배부한 뒤 현재 수량을 안내합니다. 다시 6병을 받아 최종 수량을 안내합니다. 두 안내 값과 세 번의 대입 뒤 현재 수량을 표로 작성하고, 두 print를 어디에 놓을지 판단하세요.

정답과 오답의 이유

대입 뒤 현재 값은 12→8→14이고 안내값은 8과 14입니다. 배부 직후와 추가 입고 직후에 출력해야 합니다. 모든 대입 뒤에 같은 이름을 두 번 출력하면 14·14가 되어 중간 안내를 잃습니다.

다시 확인할 질문 세 가지

1. 오른쪽 **stock**은 언제 읽나요? 그 줄을 실행하는 시점에 연결된 현재 값을 먼저 읽습니다.
2. **=**는 **==**와 같은가요? =는 대입이고 ==는 같은지 비교하는 기호입니다.

3. 나중 갠신이 이전 화면을 바꾸나요? 이미 출력한 결과는 그때의 값으로 남습니다.

개념. 변수는 프로그램이 값을 다시 찾을 수 있도록 붙인 이름입니다. location = "입구"는 왼쪽 이름에 오른쪽 값을 연결합니다. 수학에서 등식의 양변이 같다는 주장과 Python의 대입 동작을 구별합니다.

왜 필요한가. 값을 여러 번 사용할 때 이름을 붙이면 목적이 드러나고 한 곳에서 바꾸기 쉽습니다. 10/8의 표에서 location과 temperature_c라는 항목을 읽었던 경험을 코드 안의 이름으로 연결합니다.

함께 보는 예. location과 temperature_c를 출력하면 이름 자체의 글자가 아니라 그 이름이 가리키는 값이 나옵니다. print("location")처럼 따옴표를 붙이면 이름을 찾지 않고 글자 location을 보여 줍니다.

헛갈리는 경우. 한 이름이 무엇을 뜻하는지는 작성자가 정한 약속입니다. 온도를 x로 저장해도 실행될 수 있지만 업무 목적을 읽기 어렵습니다. 의미가 분명한 이름을 쓰되 이름만 보고 단위가 맞다고 단정하지 않습니다.

전체 코드

```
location = "입구"
temperature_c = 22.5
print(location)
print(temperature_c)
print("location")
```

각 줄의 뜻

줄	코드	뜻
1	location = "입구"	장소 문자열을 location이라는 이름에 연결합니다.
2	temperature_c = 22.5	숫자 22.5를 temperature_c라는 이름에 연결합니다.
3	print(location)	이름이 가리키는 장소 값을 보여 줍니다.
4	print(temperature_c)	이름이 가리키는 온도 값을 보여 줍니다.
5	print("location")	따옴표 안의 글자 location을 그대로 보여 줍니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A04_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A04_example.py
```

원본 코드의 예상 출력:

```
입구
22.5
location
```

작은 수정: [입구](#)를 [창고](#)로 바꿉니다.

수정 후 예상 출력:

```
창고
22.5
location
```

학생 적용

1. 값과 이름을 서로 다른 색으로 표시합니다.
2. location을 창고로 바꾸면 변환 출력 줄을 먼저 고릅니다.
3. 작업본에서 문자열 한 곳만 바꾸고 저장·실행합니다.
4. 변하지 않은 두 출력 줄의 이유를 씁니다.

예상 결과: 첫 출력만 창고로 바뀌고 온도와 글자 location은 유지됩니다.

확인 질문: `print(location)`과 `print("location")`은 왜 다를까요?

막혔을 때: `NameError`가 나오면 대문자·소문자와 밑줄을 비교하고 그 이름에 값을 연결한 줄이 먼저 실행되는지 확인합니다.

[활동 자료와 작성칸](#)에서 A04를 엽니다.

A04 · 직접 해보기

A04 · 변수는 값에 붙인 이름

목적: 값을 여러 번 사용할 때 이름을 붙이면 목적이 드러나고 한 곳에서 바꾸기 쉽습니다. 10/8의 표에서 location과 temperature_c라는 항목을 읽었던 경험을 코드 안의 이름으로 연결합니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A04_input.md](#)

- [A04_example.py](#)

- [A04_notes.md](#)

- [A04_example.py](#)

단계

1. 값과 이름을 서로 다른 색으로 표시합니다.

2. location을 창고로 바꾸면 변할 출력 줄을 먼저 고릅니다.
3. 작업본에서 문자열 한 곳만 바꾸고 저장·실행합니다.
4. 변하지 않은 두 출력 줄의 이유를 씁니다.

예상 결과: 첫 출력만 창고로 바뀌고 온도와 글자 location은 유지됩니다.

확인 방법: print(location)과 print("location")은 왜 다를까요?

오류 해결: NameError가 나오면 대문자·소문자와 밑줄을 비교하고 그 이름에 값을 연결한 줄이 먼저 실행되는지 확인합니다.

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 다시 대입하면 현재 값이 바뀐다 / 오류 메시지를 읽고 변수 이름 고치기

[이 교사의 다른 예와 코드 해설 열기](#)

3교시 · 문자와 숫자의 연산

13:00~13:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

A07 · 숫자 더하기와 문자열 연결

사례로 이해하기 · 운송비 계산에서 문자열과 숫자 나누기

먼저 강사의 예를 보고 같은 순서로 함께 확인합니다. 추가 문제는 필수 활동을 마친 뒤 풀어 보세요.

택배 접수표에는 상품번호 "0012", 무게 "2.5", 포장비 1000이 있습니다. CSV에서 읽은 무게 칸은 숫자처럼 보여도 문자열일 수 있습니다. 문자열은 글자 순서를 보존하고 숫자는 수량을 계산합니다. 따옴표로 감싼 "2.5"와 실수 2.5의 저장 종류가 다르므로 계산 전 의미에 맞는 변환을 해야 합니다. 상품번호는 계산용 수량이 아니므로 앞의 0을 보존한 문자열을 유지합니다. 같은 모양의 숫자라도 변환 여부는 업무 목적에 따라 결정합니다.

수업용 요금은 무게 1kg당 2000원에 포장비 1000원을 더하는 것으로 정합니다. 반올림·최소요금 등 실제 운송 규칙은 이번 합성 예에 포함하지 않습니다. "2.5"를 실수 2.5로 읽으면 $2.5 \times 2000 + 1000 = 6000$ 원입니다. 먼저 곱셈으로 무게 요금을 계산하고 포장비를 더합니다. 문자열 "2"와 "5"를 +로 붙이면 "25"가 되어 무게 7이 아닙니다. 문자열을 이어 붙이는 연결과 수치 덧셈을 같은 기호가 다른 종류에 적용된 사례로 비교합니다.

값 추적표는 원문 "2.5"·저장 종류 문자열 → 변환 값 2.5·실수 → 계산값 6000.0·실수 → 업무 표시 6000원 순서입니다. 계산 결과의 .0은 이 예의 저장·표시 방식이지 비용이 소수 원이라는 업무 확정은 아닙니다. 무게 원문이 "미확인"이라면 임의의 0kg로 바꾸지 않고 숫자 변환 전에 확인을 요청합니다. 문자열과 숫자를 +로 바로 섞으면 TypeError가 날 수 있고 "2.5"를 정수로 바로 읽으려 하면 ValueError가 납니다. 종류를 확인한 뒤 실수 변환을 고르고 상품번호까지 함께 바꾸지 않습니다.

혼자 풀어 볼 문제

복습·도전: 판매표의 주문번호 "0009", 단가 "1500", 수량 "3"을 보고 총액과 각 항목의 저장 계획을 쓰세요. 단가는 원 단위의 정수이고, 수량은 개수로 정했습니다. 배송비 2000원을 포함한 최종액도 계산하세요.

정답과 오답의 이유

주문번호는 문자열 유지, 단가와 수량은 정수로 해석합니다. 상품 총액 $1500 \times 3 = 4500$ 원, 최종액 6500원입니다. "1500"+"3"의 문자열 연결은 "15003"으로 금액 계산이 아닙니다. 주문번호를 정수 9로 바꾸면 앞 0을 잃습니다.

다시 확인할 질문 세 가지

- 1. 숫자처럼 보이면 모두 변환하나요? 식별자·등급인지 실제 계산량인지 의미를 먼저 봅니다.
- 2. 소수 문자열을 정수로 바로 읽나요? 소수가 있는 표현에는 float 같은 의미에 맞는 변환이 필요합니다.
- 3. 정상 종료면 요금도 맞나요? 단위·요금 규칙·입력 해석이 맞는지 별도 확인해야 합니다.

개념. 숫자의 +는 수치 덧셈을 하고 문자열의 +는 글자를 이어 붙입니다. 같은 모양의 3이라도 따옴표 유무에 따라 저장 자료형이 달라지며 연산의 결과도 달라집니다.

왜 필요한가. CSV의 셀을 기본 방식으로 읽으면 문자열이 된다는 10/8의 내용을 떠올립니다. 수량을 계산하려고 글자를 그대로 이어 붙이면 합계처럼 보이는 잘못된 결과가 생길 수 있습니다.

함께 보는 예. $3 + 2$ 는 5이고 "3" + "2"는 글자 32입니다. 10/8의 사번 001처럼 앞의 0을 유지해야 하는 이름표는 문자열이 적절할 수 있으므로 모든 숫자 모양을 변환하지 않습니다.

헛갈리는 경우. 문자열을 무조건 숫자로 바꾸는 것도 정답이 아닙니다. 사번을 숫자로 바꾸면 식별 표현을 잃을 수 있습니다. 먼저 의미상 수량인지, 이름표인지 판단합니다.

전체 코드

```
count = 3
text_count = "3"
print(count + 2)
print(text_count + "2")
print(type(count).__name__, type(text_count).__name__)
```

각 줄의 뜻

줄	코드	뜻
1	count = 3	숫자 3을 저장합니다.
2	text_count = "3"	글자 3을 저장합니다.
3	print(count + 2)	숫자끼리 덧셈합니다.
4	print(text_count + "2")	문자열끼리 이어 붙입니다.
5	print(type(count).__name__, type(text_count).__name__)	두 값의 자료형 이름을 확인합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A07_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A07_example.py
```

원본 코드의 예상 출력:

```
5
32
int str
```

학생 적용

1. 세 출력 줄을 예상하고 각각 계산인지 글자 연결인지 적습니다.
2. 숫자와 문자열의 +를 같은 기준으로 읽지 않았는지 비교합니다.
3. 고의 오류 `text_count + 2`에서 서로 다른 자료형을 표시합니다.
4. 수량과 사번 중 무엇을 숫자로 바꿀지 이유를 씁니다.

예상 결과: 5와 문자열 32, int와 str을 구별합니다. 혼합 +는 TypeError가 기대 결과입니다.

확인 질문: "001"에 1을 더해 다음 직원을 정하는 것이 항상 적절할까요?

막혔을 때: 화면에서 따옴표 없이 32가 보이면 type 출력도 함께 확인합니다. 표시 모양만으로 자료형을 판단하지 않습니다.

[활동 자료와 작성칸](#)에서 A07를 엽니다.

A07 · 직접 해보기

A07 · 숫자 더하기와 문자열 연결

목적: CSV의 셀을 기본 방식으로 읽으면 문자열이 된다는 10/8의 내용을 떠올립니다. 수량을 계산하려고 글자를 그대로 이어 붙이면 합계처럼 보이는 잘못된 결과가 생길 수 있습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A07_input.md](#)
- [A07_example.py](#)
- [A07_notes.md](#)
- [A07_example.py](#)

단계

1. 세 출력 줄을 예상하고 각각 계산인지 글자 연결인지 적습니다.
2. 숫자와 문자열의 +를 같은 기준으로 읽지 않았는지 비교합니다.
3. 고의 오류 `text_count + 2`에서 서로 다른 자료형을 표시합니다.
4. 수량과 사번 중 무엇을 숫자로 바꿀지 이유를 씁니다.

예상 결과: 5와 문자열 32, int와 str을 구별합니다. 혼합 +는 `TypeError`가 기대 결과입니다.

확인 방법: "001"에 1을 더해 다음 직원을 정하는 것이 항상 적절할까요?

오류 해결: 화면에서 따옴표 없이 32가 보이면 `type` 출력도 함께 확인합니다. 표시 모양만으로 자료형을 판단하지 않습니다.

수정 전 예상: _____

확인한 결과와 이유: _____

A08 · 수치 문자열을 숫자로 변환하기

개념. 형 변환은 정해진 규칙으로 다른 자료형의 값을 만드는 일입니다. `float("22.5")`는 숫자를 나타내는 문자열을 실수값으로 해석합니다. 원문 문자열이 저절로 바뀌는 것은 아니며 결과를 다른 이름에 연결할 수 있습니다.

왜 필요한가. 외부에서 읽은 온도 글자를 계산하려면 숫자로 다룰 값인지 확인하고 변환해야 합니다. 데이터의 의미가 수치라는 사실과 현재 저장 형태가 문자열이라는 사실을 연결하는 단계입니다.

함께 보는 예. `raw`와 `temperature_c`를 나누어 두면 받은 원문과 해석 결과를 비교하기 쉽습니다. 출력에서는 두 값의 모양이 같아도 자료형이 `str`과 `float`로 다릅니다.

헛갈리는 경우. `float("뜨거움")`은 수치로 해석할 수 없어 `ValueError`가 납니다. 빈칸을 0으로 바꾸는 것으로 해결하면 미측정을 실제 0°C로 바꾸게 됩니다. 값 없음 처리는 다음 활동과 다음 날에 별도로 배웁니다.

전체 코드

```
raw = "22.5"

temperature_c = float(raw)

print(raw, type(raw).__name__)

print(temperature_c, type(temperature_c).__name__)

print(temperature_c + 1)
```

각 줄의 뜻

줄	코드	뜻
1	<code>raw = "22.5"</code>	받은 원문 글자를 저장합니다.
2	<code>temperature_c = float(raw)</code>	숫자로 해석한 결과를 새 이름에 연결합니다.
3	<code>print(raw, type(raw).__name__)</code>	원문의 값과 문자열 자료형을 확인합니다.
4	<code>print(temperature_c, type(temperature_c).__name__)</code>	변환 결과와 실수 자료형을 확인합니다.
5	<code>print(temperature_c + 1)</code>	변환한 숫자에 1을 더합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A08_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A08_example.py
```

원본 코드의 예상 출력:

```
22.5 str
22.5 float
23.5
```

작은 수정: "22.5"를 "24.0"로 바꿉니다.

수정 후 예상 출력:

```
24.0 str
24.0 float
25.0
```

학생 적용

1. 원문과 변환 결과의 이름을 구별합니다.
2. 원문을 "24.0"으로 바꾸고 출력 세 줄을 예상합니다.
3. 저장·실행 뒤 값과 자료형을 대조합니다.
4. 고의 오류 "뜨거움" 사례의 원문·오류 종류·확인 질문을 기록합니다.

예상 결과: 변환 후 계산 결과는 25.0. 숫자가 아닌 글자에서는 ValueError.

확인 질문: 원문 raw를 남기는 이유는 무엇일까요?

막혔을 때: ValueError가 보이면 입력 문자열을 먼저 확인합니다. °C 같은 단위 글자가 붙었는지, 빈칸인지 구별하고 확인되지 않은 값을 만들어 채우지 않습니다.

[활동 자료와 작성칸](#)에서 A08를 엽니다.

A08 · 직접 해보기

A08 · 수치 문자열을 숫자로 변환하기

목적: 외부에서 읽은 온도 글자를 계산하려면 숫자로 다룰 값인지 확인하고 변환해야 합니다. 데이터의 의미가 수치라는 사실과 현재 저장 형태가 문자열이라는 사실을 연결하는 단계입니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A08_input.md](#)

- [A08_example.py](#)

- [A08_notes.md](#)

- [A08_example.py](#)

단계

1. 원문과 변환 결과의 이름을 구별합니다.
2. 원문을 "24.0"으로 바꾸고 출력 세 줄을 예상합니다.
3. 저장·실행 뒤 값과 자료형을 대조합니다.
4. 고의 오류 "뜨거움" 사례의 원문·오류 종류·확인 질문을 기록합니다.

예상 결과: 변환 후 계산 결과는 25.0. 숫자가 아닌 글자에서는 ValueError.

확인 방법: 원문 raw를 남기는 이유는 무엇일까요?

오류 해결: ValueError가 보이면 입력 문자열을 먼저 확인합니다. °C 같은 단위 글자가 붙었는지, 빈칸인지 구별하고 확인되지 않은 값을 만들어 채우지 않습니다.

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 연산과 괄호로 작은 계산 만들기

[이 교시의 다른 예와 코드 해설 열기](#)

4교시 · 기준에 따라 나누기

14:00~14:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

A11 · if와 else로 출력 선택하기

개념. if는 조건이 참일 때 실행할 묶음을 선택합니다. else는 그 조건이 거짓일 때 실행할 묶음입니다. 콜론 뒤의 들여쓰 줄들이 그 묶음에 속합니다.

왜 필요한가. 비교 결과를 화면에 보여 주는 단계에서 한 걸음 더 나아가, 비교에 따라 다른 일을 할 수 있습니다. 두 갈래 중 어느 쪽이 실행되는지 먼저 손으로 따라가면 코드를 읽기 쉽습니다.

함께 보는 예. 23.5는 기준 24.0을 넘지 않으므로 else 쪽의 정상이 출력됩니다. 25.0을 넣으면 if 쪽의 확인이 출력됩니다. 두 print를 모두 실행하는 코드와 차이를 비교합니다.

헛갈리는 경우. 들여쓰기를 지우면 문법 오류가 나거나 의도한 묶음이 달라질 수 있습니다. 보기 좋게 정렬하는 장식이 아니라 Python 문장의 구조입니다.

전체 코드

```
temperature_c = 23.5
limit = 24.0
if temperature_c > limit:
    print("확인")
else:
    print("정상")
```

각 줄의 뜻

줄	코드	뜻
1	temperature_c = 23.5	측정값을 정합니다.
2	limit = 24.0	수업용 비교 기준을 정합니다.
3	if temperature_c > limit:	비교가 참인지 확인하고 아래 묶음을 선택합니다.
4	print("확인")	참일 때 확인을 출력합니다.
5	else:	비교가 거짓인 경우의 묶음입니다.
6	print("정상")	거짓일 때 정상을 출력합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A11_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A11_example.py
```

원본 코드의 예상 출력:

정상

작은 수정: 23.5를 25.0로 바꿉니다.

수정 후 예상 출력:

확인

학생 적용

1. 조건식을 동그라미로 표시하고 두 갈래를 말로 설명합니다.
2. 25.0으로 바꿨을 때 실행될 줄만 표시합니다.
3. 들여쓰기를 유지한 채 값 한 곳을 수정해 실행합니다.
4. 수정한 값·실행된 갈래·출력을 노트에 연결합니다.

예상 결과: 23.5는 정상, 25.0은 확인. 양쪽 출력이 동시에 나오지 않습니다.

확인 질문: print("완료")를 들여쓰기 없이 맨 아래에 넣으면 언제 실행될까요?

막혔을 때: IndentationError가 나면 콜론 다음 줄의 공백을 원본과 비교합니다. 탭과 공백을 혼용하지 않고 제공 예의 네 칸 공백을 유지합니다.

[활동 자료와 작성칸](#)에서 A11를 엽니다.

A11 · 직접 해보기

A11 · if와 else로 출력 선택하기

목적: 비교 결과를 화면에 보여 주는 단계에서 한 걸음 더 나아가, 비교에 따라 다른 일을 할 수 있습니다. 두 갈래 중 어느 쪽이 실행되는지 먼저 손으로 따라가면 코드를 읽기 쉽습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A11_input.md](#)

- [A11_example.py](#)

- [A11_notes.md](#)

- [A11_example.py](#)

단계

1. 조건식을 동그라미로 표시하고 두 갈래를 말로 설명합니다.
2. 25.0으로 바꿨을 때 실행될 줄만 표시합니다.
3. 들여쓰기를 유지한 채 값 한 곳을 수정해 실행합니다.
4. 수정한 값·실행된 갈래·출력을 노트에 연결합니다.

예상 결과: 23.5는 정상, 25.0은 확인. 양쪽 출력이 동시에 나오지 않습니다.

확인 방법: print("완료")를 들여쓰기 없이 맨 아래에 넣으면 언제 실행될까요?

오류 해결: IndentationError가 나면 콜론 다음 줄의 공백을 원본과 비교합니다. 탭과 공백을 혼용하지 않고 제공 예의 네 칸 공백을 유지합니다.

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 비교 결과는 참 또는 거짓 / 미측정과 경계값의 세 갈래

[이 교시의 다른 예와 코드 해설 열기](#)

5교시 · 여러 기록을 반복하기

15:00~15:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

A13 · 리스트와 위치로 여러 값 담기

사례로 이해하기 · 목록의 위치와 반복 누적을 함께 추적하기

먼저 강사의 예를 보고 같은 순서로 함께 확인합니다. 추가 문제는 필수 활동을 마친 뒤 풀어 보세요.

세 배송 상자의 무게가 순서대로 2, 5, 1kg입니다. 목록은 여러 값을 정해진 순서로 묶는 방법입니다. Python의 리스트에서 첫 위치 번호는 0이고 두 번째는 1, 세 번째는 2입니다. 위치 번호는 상자 이름이 아니라 그 목록 안의 순서이므로 정렬하거나 삽입하면 달라질 수 있습니다. 길이 3은 항목 수이며 마지막 위치 번호는 2입니다. 3번째 위치라고 생각해 번호 3으로 읽으면 없는 칸을 찾는 오류가 생깁니다.

각 상자의 무게를 합하려면 반복문으로 한 항목씩 읽고 합계의 현재 값을 갱신합니다. 합계는 반복 전에 0으로 시작합니다. 첫 바퀴는 현재 합계 0에 2를 더해 2, 두 번째는 2에 5를 더해 7, 세 번째는 7에 1을 더해 8입니다. 손 추적표에는 바퀴·현재 항목·변경 전 합계·변경 후 합계를 씁니다. 반복 끝의 총합은 8kg입니다. 반복 안에서 매번 합계를 0으로 다시 만들면 2→5→1처럼 마지막 상자만 남을 수 있습니다.

또 다른 질문은 '4kg 초과인 상자가 몇 개인가'입니다. 첫 2는 조건 거짓이라 건수 0, 다음 5는 참이라 건수 1, 마지막 1은 거짓이라 건수 1입니다. 총무게 8과 초과 상자 수 1은 서로 다른 결과입니다. 누적하는 대상이 무게인지 참인 항목의 개수인지 먼저 정합니다. 반복 본문을 들여쓰는 것은 같은 일을 각 항목에 적용한다는 표시입니다. 완료 안내가 반복 안에 있으면 상자 마다 나오고 밖에 있으면 끝에 한 번 나옵니다. 목록이 비어 있으면 바퀴가 없으므로 합계의 초기값과 완료 안내의 위치를 그대로 추적합니다.

혼자 풀어 볼 문제

복습·도전: 주문별 수량 목록은 3, 0, 7, 2입니다. 총수량과 2개 초과 주문 수를 각각 손 추적표로 구하세요. 길이와 마지막 위치 번호, 빈 목록일 때 두 결과도 적으세요.

정답과 오답의 이유

총수량은 12개이고 초과 주문은 3과 7 두 건입니다. 길이 4, 마지막 번호 3입니다. 빈 목록은 초기 합계 0·건수 0을 유지합니다. 0을 오류라고 임의 삭제하거나 수량 합 12를 주문 건수로 쓰면 서로 다른 질문을 혼동한 것입니다.

다시 확인할 질문 세 가지

1. **첫 위치가 왜 0인가요?** Python 리스트가 정한 위치 번호 약속이므로 실제 목록을 그 규칙으로 읽습니다.
2. **합계 초기화는 어디에 놓나요?** 이전 바퀴 결과를 유지하도록 반복 전에 한 번 놓습니다.
3. **길이 4이면 번호 4를 읽나요?** 마지막 번호는 3이며 번호 4는 범위 밖입니다.

개념. 리스트는 여러 값을 순서대로 묶는 Python 자료구조입니다. 대괄호로 묶고 쉼표로 항목을 나눕니다. 위치 번호는 0부터 시작하며 len은 항목 개수를 돌려줍니다.

왜 필요한가. 측정값이 세 개에서 네 개로 늘어날 때 이름을 계속 만드는 대신 하나의 목록으로 다룰 수 있습니다. 10/8에 읽었던 순서 구조를 직접 작은 값 수정으로 연결합니다.

함께 보는 예. temperatures[0]은 첫 값 22.5이고 len은 3입니다. 마지막 위치 번호는 2이지 3이 아닙니다. 개수와 위치 번호를 서로 다른 질문으로 적습니다.

헛갈리는 경우. [22.5, 24.0]은 두 값의 목록이고 "22.5, 24.0"은 쉼표가 들어 있는 문자열입니다. 곁에 어떤 괄호나 따옴표가 있는지 확인합니다.

전체 코드

```
temperatures = [22.5, 24.0, 23.5]
print(temperatures[0])
print(len(temperatures))
temperatures.append(25.0)
print(len(temperatures))
print(temperatures[3])
```

각 줄의 뜻

줄	코드	뜻
1	<code>temperatures = [22.5, 24.0, 23.5]</code>	온도 세 개를 순서대로 묶습니다.
2	<code>print(temperatures[0])</code>	첫 값을 읽습니다.
3	<code>print(len(temperatures))</code>	현재 개수를 출력합니다.
4	<code>temperatures.append(25.0)</code>	목록 끝에 새 값을 덧붙입니다.
5	<code>print(len(temperatures))</code>	추가 뒤 개수를 확인합니다.
6	<code>print(temperatures[3])</code>	네 번째 값을 출력합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A13_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A13_example.py
```

원본 코드의 예상 출력:

```
22.5
3
4
25.0
```

작은 수정: `append(25.0)`를 `append(26.0)`로 바꿉니다.

수정 후 예상 출력:

```
22.5
3
4
26.0
```

학생 적용

1. 추가 전과 후의 목록을 종이에 그립니다.
2. 각 위치 번호와 개수를 따로 표시합니다.

3. 추가할 값을 26.0으로 바꾸고 출력 변화를 예상합니다.

4. 아직 없는 위치를 읽는 오류 사례를 설명합니다.

예상 결과: 추가 후 개수 4, 마지막 값 26.0. 기존 첫 값은 유지됩니다.

확인 질문: 값 하나를 수정하는 것과 append로 추가하는 것은 개수에 어떤 차이가 있나요?

막혔을 때: IndexError가 나면 읽는 시점의 목록 길이와 위치 번호를 확인합니다. 뒤에서 추가된다고 앞에서 이미 존재하는 것은 아닙니다.

[활동 자료와 작성칸](#)에서 A13를 엽니다.

A13 · 직접 해보기

A13 · 리스트와 위치로 여러 값 담기

목적: 측정값이 세 개에서 네 개로 늘어날 때 이름을 계속 만드는 대신 하나의 목록으로 다룰 수 있습니다. 10/8에 읽었던 순서 구조를 직접 작은 값 수정으로 연결합니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A13_input.md](#)

- [A13_example.py](#)

- [A13_notes.md](#)

- [A13_example.py](#)

단계

1. 추가 전과 후의 목록을 종이에 그립니다.

2. 각 위치 번호와 개수를 따로 표시합니다.

3. 추가할 값을 26.0으로 바꾸고 출력 변화를 예상합니다.

4. 아직 없는 위치를 읽는 오류 사례를 설명합니다.

예상 결과: 추가 후 개수 4, 마지막 값 26.0. 기존 첫 값은 유지됩니다.

확인 방법: 값 하나를 수정하는 것과 append로 추가하는 것은 개수에 어떤 차이가 있나요?

오류 해결: IndexError가 나면 읽는 시점의 목록 길이와 위치 번호를 확인합니다. 뒤에서 추가된다고 앞에서 이미 존재하는 것은 아닙니다.

수정 전 예상: _____

확인한 결과와 이유: _____

A14 · for로 목록을 한 항목씩 읽기

개념. for는 묶음에서 항목을 차례로 꺼내 같은 코드 묶음을 실행합니다. 반복 변수 temperature_c는 이번 차례의 값을 가리키는 이름입니다.

왜 필요한가. 여러 온도에 같은 출력이나 비교를 적용할 때 같은 코드를 복사하지 않아도 됩니다. 항목 수가 늘어나면 반복 횟수도 따라 늘어나는 관계를 관찰합니다.

함께 보는 예. 목록의 첫 22.5가 temperature_c가 되어 출력되고, 다음 차례에는 24.0, 그다음에는 23.5가 됩니다. 반복이 끝난 뒤 들여쓰기 없는 완료가 한 번 출력됩니다.

헷갈리는 경우. 반복 변수에 값을 붙인다고 원래 리스트의 모든 값이 자동 수정되는 것은 아닙니다. 오늘은 목록을 읽는 반복이며 목록 구조를 변경하는 고급 패턴은 다루지 않습니다.

전체 코드

```
temperatures = [22.5, 24.0, 23.5]
for temperature_c in temperatures:
    print(temperature_c)
print("완료")
```

각 줄의 뜻

줄	코드	뜻
1	temperatures = [22.5, 24.0, 23.5]	읽을 값의 순서를 준비합니다.
2	for temperature_c in temperatures:	한 항목씩 현재 값의 이름에 연결합니다.
3	print(temperature_c)	각 차례의 값을 출력합니다.
4	print("완료")	반복이 모두 끝난 뒤 한 번 출력합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A14_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A14_example.py
```

원본 코드의 예상 출력:

```
22.5
24.0
23.5
완료
```

작은 수정: [22.5, 24.0, 23.5]를 [22.5, 24.0, 23.5, 25.0]로 바꿉니다.

수정 후 예상 출력:

```
22.5
24.0
23.5
25.0
완료
```

학생 적용

1. 세 차례에 temperature_c가 가리키는 값을 표에 씁니다.
2. 완료가 몇 번 나올지 예상하고, 들여쓰기 위치를 보며 이유를 설명합니다.
3. 목록에 25.0을 추가하고 반복 출력 수를 비교합니다.
4. 목록이 비어 있으면 어느 줄이 실행될지 적습니다.

예상 결과: 값은 항목 수만큼 출력되고 완료는 한 번 출력됩니다.

확인 질문: 완료 출력도 네 칸 들여쓰면 무엇이 바뀔까요?

막혔을 때: 출력 횟수가 다르면 들여쓰기 범위를 먼저 확인합니다. 코드 줄 수와 실행 횟수는 다를 수 있습니다.

[활동 자료와 작성칸](#)에서 A14를 엽니다.

A14 · 직접 해보기

A14 · for로 목록을 한 항목씩 읽기

목적: 여러 온도에 같은 출력이나 비교를 적용할 때 같은 코드를 복사하지 않아도 됩니다. 항목 수가 늘어나면 반복 횟수도 따라 늘어나는 관계를 관찰합니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A14_input.md](#)

- [A14_example.py](#)

- [A14_notes.md](#)

- [A14_example.py](#)

단계

1. 세 차례에 temperature_c가 가리키는 값을 표에 씁니다.
2. 완료가 몇 번 나올지 예상하고, 들여쓰기 위치를 보며 이유를 설명합니다.
3. 목록에 25.0을 추가하고 반복 출력 수를 비교합니다.
4. 목록이 비어 있으면 어느 줄이 실행될지 적습니다.

예상 결과: 값은 항목 수만큼 출력되고 완료는 한 번 출력됩니다.

확인 방법: 완료 출력도 네 칸 들여쓰면 무엇이 바뀔까요?

오류 해결: 출력 횟수가 다르면 들여쓰기 범위를 먼저 확인합니다. 코드 줄 수와 실행 횟수는 다를 수 있습니다.

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 반복 안에서 조건 확인과 개수 세기

[이 교시의 다른 예와 코드 해설 열기](#)

6교시 · 작은 점검 프로그램

16:00~16:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

A17 · 수업용 센서 요약 완성 예제

개념. 종합 예제는 목록·반복·조건을 결합하여 각 관찰의 상태를 출력합니다. 앞에서 배운 목록·반복·조건이 코드의 어디에 쓰였는지 짚어 줍니다.

왜 필요한가. 실무에서는 한 줄의 문법을 아는 것과 여러 단계가 연결된 프로그램을 읽는 것이 다릅니다. 준비한 입력, 각 차례의 처리, 최종 출력을 나누어 표시하면 긴 코드도 추적할 수 있습니다.

함께 보는 예. S01의 22.5는 정상, S02의 25.0은 확인, S03의 None은 미측정입니다. 상태 문자열을 status라는 이름에 넣은 뒤 공통 print 한 곳에서 센서명과 함께 출력합니다.

헛갈리는 경우. 상태 표시만으로 실제 장비의 안전을 판정하지 않습니다. 24.0이라는 숫자는 수업 비교 규칙입니다. 상태가 None인지 먼저 검사하므로 미측정을 실제 0으로 채우지 않습니다.

전체 코드

```
records = [
    {"sensor_id": "S01", "temperature_c": 22.5},
    {"sensor_id": "S02", "temperature_c": 25.0},
    {"sensor_id": "S03", "temperature_c": None},
]

limit = 24.0

for record in records:
    temperature_c = record["temperature_c"]
    if temperature_c is None:
        status = "미측정"
    elif temperature_c > limit:
        status = "확인"
    else:
        status = "정상"
    print(record["sensor_id"], status)
```

각 줄의 뜻

줄	코드	뜻
1	<code>records = [</code>	관찰 목록을 시작합니다.
2	<code>{"sensor_id": "S01", "temperature_c": 22.5},</code>	첫 측정 기록입니다.
3	<code>{"sensor_id": "S02", "temperature_c": 25.0},</code>	둘째 측정 기록입니다.
4	<code>{"sensor_id": "S03", "temperature_c": None},</code>	온도가 없는 셋째 기록입니다.
5	<code>]</code>	목록을 닫습니다.
6	<code>limit = 24.0</code>	수업용 기준을 정합니다.
7	<code>for record in records:</code>	기록 한 건씩 처리합니다.
8	<code>temperature_c = record["temperature_c"]</code>	이번 기록의 온도를 읽습니다.
9	<code>if temperature_c is None:</code>	값 없음부터 확인합니다.
10	<code>status = "미측정"</code>	값이 없을 때의 상태를 정합니다.
11	<code>elif temperature_c > limit:</code>	값이 있으면 초과 여부를 묻습니다.
12	<code>status = "확인"</code>	초과한 경우의 상태를 정합니다.
13	<code>else:</code>	앞의 두 경우가 아닐 때의 묶음입니다.
14	<code>status = "정상"</code>	나머지 측정의 상태를 정합니다.
15	<code>print(record["sensor_id"], status)</code>	이번 센서와 판정 상태를 출력합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A17_sensor_summary.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A17_sensor_summary.py
```

원본 코드의 예상 출력:

S01 정상
S02 확인
S03 미측정

작은 수정: `limit = 24.0`를 `limit = 25.0`로 바꿉니다.

수정 후 예상 출력:

S01 정상
S02 정상
S03 미측정

학생 적용

1. 각 기록의 입력·조건·예상 상태를 작성합니다.
2. `limit`만 25.0으로 바꾸고 달라질 기록을 고릅니다.
3. 전체 파일을 저장·실행하고 세 출력 줄을 대조합니다.
4. 미측정 처리와 기준 비교의 실행 순서를 짝에게 설명합니다.

예상 결과: 기준을 25.0으로 바꾸면 S02는 같은 값이므로 정상이며 S03은 계속 미측정입니다.

확인 질문: 기준만 바꿨는데 미측정 상태가 달라진다면 무엇을 점검해야 할까요?

막혔을 때: 한꺼번에 여러 곳을 수정하지 않습니다. 원본과 작업본을 나란히 보고 지정한 기준 한 곳부터 다시 비교합니다.

[활동 자료와 작성칸](#)에서 A17를 엽니다.

A17 · 직접 해보기

A17 · 수업용 센서 요약 완성 예제

목적: 실무에서는 한 줄의 문법을 아는 것과 여러 단계가 연결된 프로그램을 읽는 것이 다릅니다. 준비한 입력, 각 차례의 처리, 최종 출력을 나누어 표시하면 긴 코드도 추적할 수 있습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A17_input.md](#)

- [A17_sensor_summary.py](#)

- [A17_notes.md](#)

- [A17_sensor_summary.py](#)

단계

1. 각 기록의 입력·조건·예상 상태를 작성합니다.
2. `limit`만 25.0으로 바꾸고 달라질 기록을 고릅니다.

- 3. 전체 파일을 저장·실행하고 세 출력 줄을 대조합니다.
 - 4. 미측정 처리와 기준 비교의 실행 순서를 짝에게 설명합니다.
- 예상 결과:** 기준을 25.0으로 바꾸면 S02는 같은 값이므로 정상이며 S03은 계속 미측정입니다.
- 확인 방법:** 기준만 바꿨는데 미측정 상태가 달라진다면 무엇을 점검해야 할까요?

오류 해결: 한꺼번에 여러 곳을 수정하지 않습니다. 원본과 작업본을 나란히 보고 지정한 기준 한 곳부터 다시 비교합니다.

수정 전 예상: _____
 확인한 결과와 이유: _____

A18 · 새 입력 검수와 실행 인계

개념. 프로그램 사용 결과는 파일·입력·명령·출력·판단 근거를 함께 남길 때 다른 사람이 확인할 수 있습니다. 잘 실행되었다는 한 문장보다 어떤 사례를 확인했는지가 중요합니다.

왜 필요한가. 새 입력에서 경계값과 미측정을 다시 확인하면 예제 답을 외운 것인지 원리를 이해했는지 볼 수 있습니다. 실행하지 못한 경우에는 예상·강사 관찰·개인 실행을 정확히 구별합니다.

함께 보는 예. 새 입력은 S04의 24.0, S05의 0.0, S06의 None입니다. 기준은 24.0 초과입니다. 정상·정상·미측정으로 예상하고, 기존 A17 작업본의 기록 목록만 바꾸는 수정안을 작성합니다.

헛갈리는 경우. 과거 A17 출력 화면을 새 입력의 결과라고 제출하면 재현 근거가 맞지 않습니다. 파일을 저장한 시점과 실행한 경로, 새 입력값을 함께 확인해야 합니다.

제공 입력

새 관찰: S04=24.0, S05=0.0, S06=None. 기준 24.0 초과.

인계 양식: 실행 파일 / 바꾼 입력 / 실행 명령 / 예상 출력 / 실제 확인 방식 / 남은 질문.

학생 적용

- 1. 새 관찰 세 개의 예상 상태와 근거를 적습니다.
- 2. A17 구조를 재사용해 work/A18_final.py의 기록 목록을 새 자료로 바꿉니다.
- 3. 같은 기준으로 실행하거나 제공 출력과 수동 대조하고 확인 방식을 기록합니다.
- 4. 짝이 같은 파일을 찾고 결과를 설명할 수 있게 인계 세 문장을 씁니다.

예상 결과: S04 정상, S05 정상, S06 미측정. 0.0과 None이 다르게 처리됩니다.

확인 질문: 내일 파일에서 읽은 "24.0"을 그대로 이 코드에 넣으면 무엇이 추가로 필요할까요?

막혔을 때: 새 파일이 실행되지 않으면 먼저 원본 A17을 실행해 실행 환경 문제인지, 새로 수정한 코드의 문제인지 확인합니다. 원본까지 실패하면 환경 단서부터 확인합니다.

[활동 자료와 작성칸](#)에서 A18를 엽니다.

공식 참고자료

- [introduction.html](#)
- [controlflow.html](#)
- [functions.html](#)

공식 동작은 2026-09-26 확인했습니다. 자체 합성 사례로 설명하며 현재 교실 환경이나 학습 효과를 실측한 결과는 아닙니다.

A18 · 직접 해보기

A18 · 새 입력 검수와 실행 인계

추가 원문·작업 파일:

- [A18_input.md](#)
- [A18_notes.md](#)
- [A18_final.py](#)

목적: 새 입력에서 경계값과 미측정을 다시 확인하면 예제 답을 외운 것인지 원리를 이해했는지 볼 수 있습니다. 실행하지 못한 경우에는 예상·강사 관찰·개인 실행을 정확히 구별합니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A18_input.md](#)
- [A18_notes.md](#)

단계

1. 새 관찰 세 개의 예상 상태와 근거를 적습니다.
2. A17 구조를 재사용해 work/A18_final.py의 기록 목록을 새 자료로 바꿉니다.
3. 같은 기준으로 실행하거나 제공 출력과 수동 대조하고 확인 방식을 기록합니다.
4. 짝이 같은 파일을 찾고 결과를 설명할 수 있게 인계 세 문장을 씁니다.

예상 결과: S04 정상, S05 정상, S06 미측정. 0.0과 None이 다르게 처리됩니다.

확인 방법: 내일 파일에서 읽은 "24.0"을 그대로 이 코드에 넣으면 무엇이 추가로 필요할까요?

오류 해결: 새 파일이 실행되지 않으면 먼저 원본 A17을 실행해 실행 환경 문제인지, 새로 수정한 코드의 문제인지 확인합니다. 원본까지 실패하면 환경 단서부터 확인합니다.

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 관찰 한 건과 점검표 구조

[이 교사의 다른 예와 코드 해설 열기](#)

오늘 배운 내용 확인

확인할 것	배점	충족 기준
배운 내용 설명	20점	사용한 개념과 선택한 이유를 예로 설명한다.
결과물 완성	40점	오늘 필수 활동의 파일·표·기획 내용을 빠짐없이 완성한다.
결과 확인	30점	예상과 결과를 비교하고 다른 입력 한 가지로 다시 확인한다.

저장과 전달	10점	파일 이름과 실행·읽는 방법을 함께 적어 다른 사람이 확인할 수 있다.
--------	-----	---

각 항목: 모두 충족하면 배점 전부, 일부 충족하면 배점의 절반, 근거가 없으면 0점. 오류를 발견하고 고친 기록도 확인 근거로 사용합니다.

남길 결과: A17_notes.md, A17_sensor_summary.py / A18_notes.md, A18_final.py

세 가지 확인 질문

1. "22.5"라는 글자와 숫자 22.5를 더할 때 무엇이 달라지나요?

내 설명: _____
2. 오늘 필수 활동에서 값을 한 곳 바꾼 예를 고르세요. 바꾸기 전 예상과 확인한 결과를 설명해 보세요.

내 설명: _____
3. 마지막 결과물의 파일 이름과 읽거나 실행하는 방법을 적어 보세요. 다른 사람이 확인할 수 있도록 한 문장씩 안내합니다.

내 설명: _____

더 해보기 · 추가 확인 문제

구간별 확인 질문

1. 오류 줄 번호가 마지막 줄을 가리키면 그 줄만 보면 충분할까요?

2. 오류가 사라졌다는 것만으로 올바른 수정이라고 할 수 있나요?

3. 코드가 종료 코드 0으로 끝났다면 평균의 업무 의미도 보장되나요?

4. None에 직접 > 24.0을 적용하면 어떻게 될까요?

5. 한 번의 print인데 세 항목을 검사할 수 있는 이유는 무엇인가요?

6. 내일 파일에서 읽은 "24.0"을 그대로 이 코드에 넣으면 무엇이 추가로 필요할까요?

문제 1 · 편집 화면·저장 파일·실행 결과의 세 상태

복습·도전: 편집기는 work/check.py에 '검사 B'를 보여 줍니다. 실행 명령은 code/check.py를 가리키고 출력은 '검사 A'입니다. 확인된 사실 세 가지와 다음 확인 동작 두 가지를 적으세요. 새 설치나 임의 파일 삭제는 필요하지 않습니다.

정답·해설: 편집한 파일과 실행한 파일의 위치가 다르며 화면에는 A가 나왔습니다. work 파일을 저장했는지 확인하고 같은 work 주소를 실행합니다. 파일이 다르다는 증거는 있지만 저장 여부는 아직 미확인입니다. 'Python 고장'은 증거 없는 원인 단정입니다.

문제 2 · 재고 갱신을 한 줄씩 추적하기

복습·도전: 물 12병을 준비하고 4병을 배부한 뒤 현재 수량을 안내합니다. 다시 6병을 받아 최종 수량을 안내합니다. 두 안내 값과 세 번의 대입 뒤 현재 수량을 표로 작성하고, 두 print를 어디에 놓을지 판단하세요.

정답·해설: 대입 뒤 현재 값은 12→8→14이고 안내값은 8과 14입니다. 배부 직후와 추가 입고 직후에 출력해야 합니다. 모든 대입 뒤에 같은 이름을 두 번 출력하면 14·14가 되어 중간 안내를 잃습니다.

문제 3 · 운송비 계산에서 문자열과 숫자 나누기

복습·도전: 판매표의 주문번호 "0009", 단가 "1500", 수량 "3"을 보고 총액과 각 항목의 저장 계획을 쓰세요. 단가는 원 단위의 정수이고, 수량은 개수로 정했습니다. 배송비 2000원을 포함한 최종액도 계산하세요.

정답·해설: 주문번호는 문자열 유지, 단가와 수량은 정수로 해석합니다. 상품 총액 1500×3=4500원, 최종액 6500원입니다. "1500"+"3"의 문자열 연결은 "15003"으로 금액 계산이 아닙니다. 주문번호를 정수 9로 바꾸면 앞 0을 잃습니다.

문제 4 · 이상과 초과를 업무 문장으로 읽기

복습·도전: 대여 정책은 '잔여 수량이 5개 이하이면 보충 요청'입니다. 4, 5, 6, 미확인 네 입력의 결과와 비교 기호를 정하세요. 미확인은 보충 확정 대신 확인 요청으로 처리합니다.

정답·해설: 기호는 <=이며 4와 5는 보충 요청, 6은 요청 없음, 미확인은 확인 요청입니다. <를 고르면 경계 5를 놓칩니다. 미확인을 0으로 채우면 수량을 실제 관찰한 것처럼 처리하는 오답입니다.

문제 5 · 목록의 위치와 반복 누적을 함께 추적하기

복습·도전: 주문별 수량 목록은 3, 0, 7, 2입니다. 총수량과 2개 초과 주문 수를 각각 손 추적표로 구하세요. 길이와 마지막 위치 번호, 빈 목록일 때 두 결과도 적으세요.

정답·해설: 총수량은 12개이고 초과 주문은 3과 7 두 건입니다. 길이 4, 마지막 번호 3입니다. 빈 목록은 초기 합계 0·건수 0을 유지합니다. 0을 오류라고 임의 삭제하거나 수량 합 12를 주문 건수로 쓰면 서로 다른 질문을 혼동한 것입니다.

문제 6 · 한 기록의 이름표를 지키며 상태표 만들기

복습·도전: 세 물품 P01 수량 0, P02 수량 미확인, P03 수량 2를 대상으로 '2개 이하이면 주문 요청' 상태표를 새로 설계하세요. 물품번호와 수량의 이름표, 미확인 갈래를 먼저 정하고 세 줄을 작성하세요.

정답·해설: 이름표는 예를 들어 product_id와 quantity입니다. P01·P03은 주문 요청, P02는 수량 확인입니다. 0은 실제 수량이라 포함하고 None은 비교 전에 처리합니다. temperature_c 키를 그대로 쓰는 답은 새 업무의 항목 뜻을 맞추지 못한 것입니다.